

University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 8

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 17/10/2026

Lab Task 8: Page Replacement Algorithms

(FIFO vs LRU — Comparison)

CSE 402: Operating System Lab | Student ID: 23101143

---

1. Objective

To implement the FIFO and LRU page replacement algorithms in Python, run them on the same reference strings, and compare their page faults and hit ratios to decide which algorithm performs better.

2. Theory

When a process references a page that is not in memory (a page fault) and all frames are full, the operating system must choose a victim page to replace. The quality of this choice decides how many page faults occur. Hit Ratio = (Number of Hits / Total References) × 100.

- FIFO (First-In First-Out): replaces the page that has been in memory the longest. It is very simple (a queue), but it ignores how often or how recently a page is used, so it may evict a heavily used page. It can suffer from Belady's anomaly, where more frames cause more page faults.
- LRU (Least Recently Used): replaces the page that has not been used for the longest time, assuming recent past use predicts near-future use (locality of reference). It never suffers from Belady's anomaly, but needs extra bookkeeping (timestamps or a stack) to track recency.

3. Problem Data

Test	Reference String	Frames
Test 1	7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (20 references)	3
Test 2	1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (12 references)	3 and 4

Table 1: Input data (Test 1 = main comparison, Test 2 = Belady's anomaly check)

4. Implementation Summary

- fifo(pages, frames): keeps frame slots and a pointer to the oldest page; on a fault it overwrites the slot at the pointer and moves the pointer circularly.
- lru(pages, frames): records the last-used time of every page; on a fault with full frames it replaces the page with the smallest last-used time.
- show(): prints the frame-by-frame trace, page faults, hits, and hit ratio.

- Test 1 compares both algorithms with 3 frames; Test 2 repeats with 3 and 4 frames to check Belady's anomaly.

5. Step-by-Step Trace (Test 1, 3 Frames)

F = page fault (red), H = hit (green). Bold values show the page that was just loaded.

5.1 FIFO

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7
Frame 2	-	**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0
Frame 3	-	-	**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**
Result	**F**	**F**	**F**	**F**	**H**	**F**	**F**	**F**	**F**	**F**	**F**	**H**	**H**	**F**	**F**	**H**	**H**	**F**	**F**	**F**

Table 2: FIFO trace — 15 page faults

5.2 LRU

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1
Frame 2	-	**0**	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0
Frame 3	-	-	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7
Result	**F**	**F**	**F**	**F**	**H**	**F**	**H**	**F**	**F**	**F**	**F**	**H**	**H**	**F**	**H**	**F**	**H**	**F**	**H**	**H**

Table 3: LRU trace — 12 page faults

6. Results and Comparison

Metric	FIFO	LRU	Better
Page Faults	15	12	**LRU**
Page Hits	5	8	**LRU**
Hit Ratio	25.0%	40.0%	**LRU**
Fault Rate	75.0%	60.0%	**LRU**

Table 4: FIFO vs LRU on Test 1 (20 references, 3 frames)

7. Belady's Anomaly Test (Test 2)

Frames	FIFO Faults	LRU Faults	Fewer Faults
3	9	10	**FIFO**
4	10	8	**LRU**

Table 5: Page faults for the string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

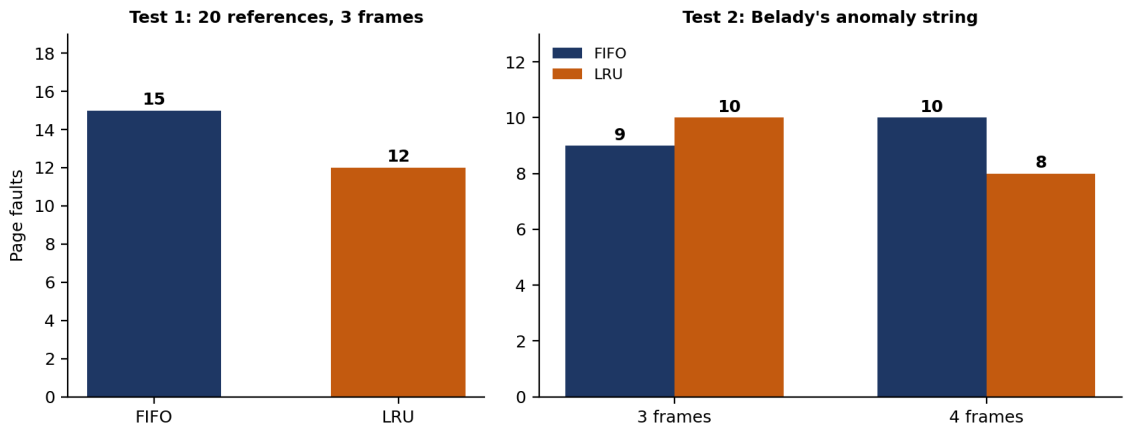


Figure 1: Page faults — FIFO vs LRU (Test 1 and Test 2)

8. Program Output

```
FIFO (frames = 3)
Ref   :  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
Frame1:  7  7  7  2  2  2  2  4  4  4  0  0  0  0  0  0  0  7  7  7
Frame2:  -  0  0  0  0  3  3  3  2  2  2  2  2  1  1  1  1  1  0  0
Frame3:  -  -  1  1  1  1  0  0  0  3  3  3  3  3  2  2  2  2  2  1
Result:  F  F  F  F  H  F  F  F  F  F  F  H  H  F  F  H  H  F  F  F
Page Faults = 15 | Hits = 5 | Hit Ratio = 25.0%

LRU (frames = 3)
Ref   :  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
Frame1:  7  7  7  2  2  2  2  4  4  4  0  0  0  1  1  1  1  1  1  1
Frame2:  -  0  0  0  0  0  0  0  0  3  3  3  3  3  3  0  0  0  0  0
Frame3:  -  -  1  1  1  3  3  3  2  2  2  2  2  2  2  2  2  7  7  7
Result:  F  F  F  F  H  F  H  F  F  F  F  H  H  F  H  F  H  F  H  H
Page Faults = 12 | Hits = 8 | Hit Ratio = 40.0%

--- Comparison (Test 1) ---
Metric            FIFO      LRU
Page Faults       15        12
Page Hits         5         8
Hit Ratio (%)     25.0      40.0

--- Belady's Anomaly Test ---
Reference string: [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
Frames    FIFO      LRU
3         9         10
4         10        8

--- Result ---
LRU is better: 12 faults vs 15 for FIFO (Test 1).
```

Figure 2: Console output of the FIFO vs LRU program

9. Discussion

On Test 1, FIFO caused 15 page faults (hit ratio 25%), while LRU caused only 12 (40%) — 3 fewer faults, i.e. 20% fewer than FIFO. The reason is visible in the traces: at the 6th reference (page 3), FIFO evicted page 0 because it was the oldest, even though page 0 had just been used; the very next reference to page 0 then caused another fault. LRU instead evicted page 1, the least recently used, so page 0 stayed in memory and the next reference to it was a hit.

Test 2 shows Belady's anomaly. With FIFO, increasing the frames from 3 to 4 raised the faults from 9 to 10, which is the wrong direction. LRU behaved correctly: faults fell from 10 to 8. Note that with 3 frames FIFO was slightly better on this particular string (9 vs 10), so no algorithm wins on every possible string — but LRU's behaviour is consistent and predictable.

Criterion	FIFO	LRU	Better
Page faults (Test 1)	15	12	**LRU**
Belady's anomaly	Can occur	Never occurs	**LRU**
Uses locality of reference	No (age only)	Yes (recency)	**LRU**
Implementation cost	Very low (queue/pointer)	Higher (timestamps/stack)	**FIFO**

Table 6: FIFO vs LRU at a glance

Verdict: LRU is the better algorithm — it gave fewer page faults (12 vs 15), a higher hit ratio (40% vs 25%), and does not suffer from Belady's anomaly. FIFO's only advantage is its simplicity.

10. Conclusion

FIFO and LRU page replacement were implemented and compared on the same reference strings. LRU performed better, with 12 page faults against 15 for FIFO on the main test, and it avoided the Belady's anomaly that FIFO showed on the second test. FIFO is easier and cheaper to implement, but its blind replacement of the oldest page makes it less efficient. Therefore LRU is preferred when the extra bookkeeping cost is acceptable.

11. Appendix: Python Source Code

The complete Python implementation used for this lab:

```
# Page Replacement Algorithms: FIFO vs LRU
# CSE 402: Operating System Lab | Lab Task 8

def fifo(pages, frames):
    mem = [None] * frames      # frame slots
    ptr = 0                    # slot holding the oldest page
    faults = 0
    trace = []
    for p in pages:
        if p in mem:
            status = "H"
        else:
            status = "F"
            faults += 1
            mem[ptr] = p                    # replace the oldest page
            ptr = (ptr + 1) % frames
        trace.append((p, mem[:], status))
    return faults, trace


def lru(pages, frames):
    mem = [None] * frames
    last_used = {}             # page -> time of most recent use
    faults = 0
    trace = []
    for t, p in enumerate(pages):
        if p in mem:
            status = "H"
        else:
            status = "F"
            faults += 1
            if None in mem:                 # free frame available
                mem[mem.index(None)] = p
            else:                           # replace least recently used page
                victim = min(mem, key=lambda x: last_used[x])
                mem[mem.index(victim)] = p
        last_used[p] = t
        trace.append((p, mem[:], status))
    return faults, trace


def show(name, pages, frames, faults, trace):
    hits = len(pages) - faults
    print(f"\n{name} (frames = {frames})")
    print("Ref   : " + " ".join(f"{p:>2}" for p in pages))
    for f in range(frames):
        row = [("-" if t[1][f] is None else t[1][f]) for t in trace]
        print(f"Frame{f + 1}: " + " ".join(f"{x:>2}" for x in row))
    print("Result: " + " ".join(f"{t[2]:>2}" for t in trace))
    print(f"Page Faults = {faults} | Hits = {hits} | Hit Ratio = {hits / len(pages) *
100:.1f}%")


# ---------- Test 1: textbook reference string, 3 frames ----------
pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
frames = 3

fifo_faults, fifo_trace = fifo(pages, frames)
lru_faults, lru_trace = lru(pages, frames)

show("FIFO", pages, frames, fifo_faults, fifo_trace)
show("LRU", pages, frames, lru_faults, lru_trace)
```

```
n = len(pages)
print("\n--- Comparison (Test 1) ---")
print(f"{'Metric':<18}{'FIFO':<10}{'LRU':<10}")
print(f"{'Page Faults':<18}{fifo_faults:<10}{lru_faults:<10}")
print(f"{'Page Hits':<18}{n - fifo_faults:<10}{n - lru_faults:<10}")
print(f"{'Hit Ratio (%)':<18}{(n - fifo_faults) / n * 100:<10.1f}{(n - lru_faults) / n *
100:<10.1f}")

# ---------- Test 2: Belady's anomaly string, 3 vs 4 frames ----------
belady = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
print("\n--- Belady's Anomaly Test ---")
print(f"Reference string: {belady}")
print(f"{'Frames':<10}{'FIFO':<10}{'LRU':<10}")
for fr in (3, 4):
    f, _ = fifo(belady, fr)
    l, _ = lru(belady, fr)
    print(f"{fr:<10}{f:<10}{l:<10}")

# ---------- Verdict ----------
print("\n--- Result ---")
if lru_faults < fifo_faults:
    print(f"LRU is better: {lru_faults} faults vs {fifo_faults} for FIFO (Test 1).")
elif fifo_faults < lru_faults:
    print(f"FIFO is better: {fifo_faults} faults vs {lru_faults} for LRU (Test 1).")
else:
    print("Both algorithms perform equally (Test 1).")
```